

First Module

Marco Barberio

Master Degree in Computer Science

m.barberio2@studenti.unipi.it

Parallel and Distributed Systems: Paradigms and Models, Academic Year: 2025/2026

1 Design choices

For the partition mapping kernel, I adopted a multiply-shift-add hash of the form

$$h_{a,b}(x) = ((a \cdot x + b) \bmod 2^{64}) \gg (64 - k),$$

where x is a 64-bit key and $P = 2^k$ is the number of partitions. This choice is inspired by classical multiply-shift and multiply-shift-add hashing schemes, which are particularly convenient when the output range size is a power of two, and represent a good trade-off between computational efficiency and quality of data distribution [1].

I assumed that P is a power of two, i.e. $P = 2^k$. This makes the final mapping especially efficient, because the partition identifier can be obtained by extracting the top k bits of the 64-bit value $(a \cdot x + b)$. In practice, this avoids the need for a generic modulo or integer division and reduces the computation to simple arithmetic and bitwise operations, namely one multiplication, one addition, and one right shift per key.

I generate the parameters a and b pseudo-randomly from the experiment seed. This makes the mapping less dependent on arbitrary constant choices and helps obtain a more generic evaluation of the hash function. Since the generation is seed-based, the coefficients are still deterministic, ensuring reproducibility of both correctness tests and performance measurements.

2 Compiler flags and report

To build the four versions of the program, I used different compiler flags depending on the implementation goal:

- `-O3 -fno-tree-vectorize` for the baseline version, in order to disable GCC auto-vectorization and obtain a scalar reference implementation;

- `-O3 -march=native -mavx2 -mprefer-vector-width=256 -fopt-info-vec` for the auto-vectorized version, in order to enable aggressive optimization, allow AVX2 instructions, encourage the compiler to use 256-bit vectors, and print the vectorization report;
- `-O3 -mavx2 -march=native` for the manual intrinsic version, in order to compile code using AVX2 intrinsics.

For the auto-vectorized build, GCC reported the following optimization message for the mapping loop:

```
1  src/mapping_baseline.cpp:14:22: optimized: loop vectorized using 32 byte vectors
```

This message refers to the main loop of the mapping function. Therefore, GCC successfully auto-vectorized the hot loop of the plain implementation. The report indicates the use of 32-byte vectors, i.e. 256-bit SIMD vectors, which means that four 64-bit keys are processed per vector operation.

3 Intrinsics strategy

In the AVX2 version, the main `for` loop processes four 64-bit integers in parallel, since AVX2 registers are 256 bits wide and can therefore contain exactly four 64-bit values. Apart from this parallelization, the sequence of operations is the same as in the scalar version.

The only non-trivial part is the multiplication $a \cdot x$. AVX2 does not directly support a full lane-wise 64-bit integer multiplication, so this step must be reconstructed from 32-bit products. If

$$x = x_{lo} + (x_{hi} \ll 32), \quad a = a_{lo} + (a_{hi} \ll 32),$$

then the lower 64 bits of the product are computed as

$$a \cdot x \equiv (a_{lo}x_{lo}) + (a_{hi}x_{lo} + a_{lo}x_{hi}) \ll 32 \pmod{2^{64}}.$$

After this multiplication, the remaining instructions are identical to the scalar version.

4 Performance Results on node09

To evaluate performance on `node09`, I ran the three implementations (baseline, auto-vectorized, and manual AVX2) on several input sizes N , using multiple seeds for each configuration. The

benchmarking procedure also included a warmup phase: before measuring execution time, each executable runs the keys mapping function a fixed number of times on the same input. In my experiments, I used 5 warmup iterations for each run. This reduces cold-start effects and makes the measured execution time more representative of steady-state performance.

For each configuration, I then summarized the results using the median time and median throughput, and I report the variability across runs using the standard deviation.

N	Baseline [μ s]	Autovec [μ s]	AVX2 [μ s]	Speedup autovec	Speedup AVX2	Throughput autovec [Melem/s]	Throughput AVX2 [Melem/s]
100	0.26 ± 0.02	0.24 ± 0.01	0.23 ± 0.01	1.08x	1.05x	416.67 ± 19.44	434.78 ± 24.98
500	0.61 ± 0.02	0.53 ± 0.04	0.55 ± 0.01	1.17x	1.11x	941.62 ± 52.90	907.44 ± 21.88
1000	1.06 ± 0.01	0.92 ± 0.08	0.93 ± 0.01	1.15x	1.14x	1084.60 ± 74.67	1072.96 ± 14.34
5000	4.71 ± 0.04	4.07 ± 0.04	4.10 ± 0.02	1.16x	1.15x	1229.11 ± 12.38	1220.11 ± 5.17
10000	9.31 ± 0.61	8.01 ± 0.08	8.05 ± 0.62	1.17x	1.15x	1249.22 ± 12.82	1241.46 ± 76.31
50000	45.97 ± 0.10	38.98 ± 4.25	39.73 ± 0.06	1.18x	1.16x	1282.61 ± 102.77	1258.34 ± 2.01
100000	91.86 ± 3.43	79.43 ± 3.69	79.49 ± 0.55	1.16x	1.15x	1258.99 ± 51.15	1258.05 ± 8.56
500000	487.70 ± 20.50	419.23 ± 7.44	429.69 ± 4.61	1.16x	1.14x	1192.68 ± 20.11	1163.62 ± 12.35
1000000	1214.79 ± 18.10	1098.45 ± 33.87	1132.15 ± 20.54	1.11x	1.07x	910.37 ± 26.33	883.28 ± 15.60
5000000	6162.85 ± 99.89	5541.63 ± 53.15	5824.44 ± 100.44	1.10x	1.05x	902.26 ± 8.49	858.45 ± 15.23
10000000	12475.90 ± 308.81	11406.80 ± 56.02	11649.40 ± 156.38	1.09x	1.07x	876.67 ± 4.30	858.41 ± 11.73
50000000	59916.50 ± 1067.45	54163.10 ± 847.00	56207.40 ± 1064.24	1.10x	1.07x	923.14 ± 14.11	889.56 ± 17.15

Table 1: Performance comparison between baseline, auto-vectorized, and AVX2 implementations on `node09` with $P = 2^{16}$.

Table 1 shows that both vectorized implementations are faster than the scalar baseline. The highest speedups are observed for medium input sizes, where the auto-vectorized version reaches about 1.16 $\times$ - 1.18 $\times$, while the AVX2 version reaches about 1.14 $\times$ - 1.16 $\times$. In the same range, throughput is around 1.2 - 1.3 Gelem/s.

For very small inputs, the speedup is limited because in this regime, the amount of useful work is too small for SIMD parallelism to provide a large benefit.

For medium input sizes, the vectorized implementations achieve the best results. In this region, the computation is large enough for SIMD execution to be effective, while the working set is still handled efficiently by the cache hierarchy. This is why both speedup and throughput reach their best values in the middle part of the table.

For large inputs, the speedup decreases and throughput drops from its peak value. This suggests that performance becomes increasingly limited by memory traffic and cache effects rather than by pure computation. Once the arrays become large, the cost of reading the input and writing the

output reduces the relative advantage of SIMD execution.

Finally, the auto-vectorized version is often slightly faster than the manual AVX2 one. A likely reason is that, in the intrinsic implementation, the 64-bit multiplication must be reconstructed from multiple 32-bit partial products, introducing extra instructions.

5 CUDA Performance Analysis

The benchmark procedure used to evaluate the CUDA implementation was the same as for the CPU versions. From Table 2, we observe that the kernel execution time is very small compared to the total execution time. This means that most of the execution time is spent on memory transfers between host and device (H2D and D2H), not on the computation itself.

This behavior is also reflected in the speedup values. The kernel speedup reaches very high values (up to about $60\times$), meaning that the GPU computes the mapping much faster than the CPU. However, the total speedup is always below $1\times$, meaning that the overall CUDA implementation is slower than the CPU version when transfer times are included.

This happens because the mapping function is a memory-bound kernel with very low computational intensity. Each element requires only a few arithmetic and bitwise operations, so the GPU can compute it very quickly, but the cost of transferring data to and from the GPU dominates the total execution time.

N	baseline [μs]	Total time [μs]	Kernel time [μs]	H2D time [μs]	D2H time [μs]	Speedup total	Speedup kernel	Throughput total [Melem/s]	Throughput kernel [Melem/s]
100	0.26 ± 0.02	342.43 ± 11.59	14.34 ± 1.22	18.40 ± 1.18	17.60 ± 0.96	0.00x	0.02x	0.29 ± 0.01	6.98 ± 0.47
500	0.61 ± 0.02	349.83 ± 7.71	15.36 ± 4.87	19.46 ± 0.76	18.56 ± 0.68	0.00x	0.04x	1.43 ± 0.03	32.55 ± 4.97
1000	1.06 ± 0.01	353.55 ± 8.68	16.38 ± 0.97	19.62 ± 0.66	19.20 ± 0.79	0.00x	0.07x	2.83 ± 0.07	61.04 ± 3.62
5000	4.71 ± 0.04	355.56 ± 14.62	15.36 ± 1.57	24.00 ± 1.41	24.10 ± 1.58	0.01x	0.31x	14.06 ± 0.54	325.52 ± 27.90
10000	9.31 ± 0.61	361.89 ± 14.76	16.10 ± 1.44	28.19 ± 0.58	29.02 ± 1.55	0.03x	0.58x	27.63 ± 1.02	621.27 ± 46.67
50000	45.97 ± 0.10	477.84 ± 10.40	18.02 ± 1.05	64.58 ± 0.73	97.79 ± 0.89	0.10x	2.56x	104.64 ± 2.18	2775.31 ± 157.90
100000	91.86 ± 3.43	594.50 ± 8.03	18.02 ± 0.70	105.70 ± 1.05	176.00 ± 4.04	0.15x	5.12x	168.21 ± 2.26	5550.62 ± 223.47
500000	487.70 ± 20.50	1639.95 ± 21.73	30.50 ± 1.69	407.87 ± 2.76	619.14 ± 6.56	0.30x	16.04x	304.89 ± 4.03	16395.59 ± 1014.69
1000000	1214.79 ± 18.10	3094.03 ± 52.02	52.64 ± 2.44	849.06 ± 22.88	1140.64 ± 18.64	0.39x	23.32x	323.20 ± 5.46	18996.96 ± 899.63
5000000	6162.85 ± 99.89	13135.50 ± 210.59	149.57 ± 2.57	4944.00 ± 209.79	5453.60 ± 81.37	0.47x	41.14x	380.65 ± 6.23	33429.61 ± 565.17
10000000	12475.90 ± 308.81	23365.50 ± 842.57	244.00 ± 1.80	9814.85 ± 844.57	10511.30 ± 72.35	0.53x	51.09x	427.98 ± 14.21	40983.61 ± 303.85
50000000	59916.50 ± 1067.45	113584.00 ± 2194.76	1020.64 ± 1.49	56826.80 ± 162.17	52353.30 ± 2173.90	0.54x	59.89x	440.20 ± 8.10	48988.87 ± 71.66

Table 2: CUDA implementantion performance on node09 with $P = 2^{16}$ and block size=256. Total time includes host-device transfers.

In conclusion, the GPU implementation achieves very high kernel throughput, but the overall performance is dominated by host-device transfer time. Therefore, performance is limited by

memory transfers rather than computation.

References

- [1] Christopher Genovese and Alex Reinhart. *Hashing — 36-750 Statistical Computing*. <https://36-750.github.io/methods/hashing/>. Accessed: 2026-03-23.